

PYTHON PACKAGES

```
from collections import Counter
from string import punctuation
```

```
def load_text(input_file):
    """Load text from a file.
```

Parameters

```
input_file : str
    Path to text file.
```

Returns

```
str
    Contents of the text
```

Examples

```
>>> fpath = "text.txt"
>>> load_text(fpath)
"""
with open(input_file, "r") as file:
    text = file.read()
return text
```



```
def count_words(text):
    """Count the words in a string.
```

String is lowercased and punctuation is removed before words are counted.

Parameters

```
str
    Text to count words in.
```

Returns

collections.Counter
A Counter-like object where keys are words and values are their counts.

```
count_words("Hello, world!")
```

```
"""
text = text.lower()
for p in punctuation:
    text = text.replace(p, "")
words = text.split()
return Counter(words)
```

TOMAS BEUZEN
TIFFANY TIMBERS



CRC Press

Taylor & Francis Group

A CHAPMAN & HALL BOOK

Python Packages

Chapman & Hall/CRC

The Python Series

About the Series

Python has been ranked as the most popular programming language, and it is widely used in education and industry. This book series will offer a wide range of books on Python for students and professionals. Titles in the series will help users learn the language at an introductory and advanced level, and explore its many applications in data science, AI, and machine learning. Series titles can also be supplemented with Jupyter notebooks.

Image Processing and Acquisition using Python, Second Edition

Ravishankar Chityala, Sridevi Pudipeddi

Python Packages

Tomas Beuzen and Tiffany Timbers

For more information about this series please visit: <https://www.crcpress.com/Chapman-HallCRC/book-series/PYTH>

Python Packages

Tomas Beuzen
Tiffany Timbers



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business
A CHAPMAN & HALL BOOK

First edition published 2022
by CRC Press
6000 Broken Sound Parkway NW, Suite 300, Boca Raton, FL 33487-2742

and by CRC Press
4 Park Square, Milton Park, Abingdon, Oxon, OX14 4RN

CRC Press is an imprint of Taylor & Francis Group, LLC

© 2022 Tomas Beuzen and Tiffany Timbers

Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, access www.copyright.com or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. For works that are not available on CCC please contact mpkbookspermissions@tandf.co.uk

Trademark notice: Product or corporate names may be trademarks or registered trademarks and are used only for identification and explanation without intent to infringe.

ISBN: 978-1-032-03825-4 (hbk)
ISBN: 978-1-032-02944-3 (pbk)
ISBN: 978-1-003-18925-1 (ebk)

DOI: [10.1201/9781003189251](https://doi.org/10.1201/9781003189251)

Publisher's note: This book has been prepared from camera-ready copy provided by the authors.

To you, the Reader.

Never stop learning. You are capable of anything.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

List of Figures	xi
List of Tables	xiii
Preface	xv
About the authors	xix
1 Introduction	1
1.1 Why you should create packages	2
2 System setup	3
2.1 The command-line interface	3
2.2 Installing software	3
2.2.1 Installing Python	3
2.2.2 Install packaging software	4
2.3 Register for a PyPI account	5
2.4 Set up Git and GitHub	5
2.5 Python integrated development environments	6
2.5.1 Visual Studio Code	7
2.5.2 JupyterLab	8
2.5.3 RStudio	10
2.6 Developing with Docker	11
2.6.1 Docker with Visual Studio Code	12
2.6.2 Docker with JupyterLab	14
3 How to package a Python	17
3.1 Counting words in a text file	17
3.1.1 Developing our code	17
3.1.2 Turning our code into functions	20
3.2 Package structure	22
3.2.1 A brief introduction	22
3.2.2 Creating a package structure	23
3.3 Put your package under version control	26
3.3.1 Set up local version control	26
3.3.2 Set up remote version control	27
3.4 Packaging your code	30

3.5	Test drive your package code	32
3.5.1	Create a virtual environment	32
3.5.2	Installing your package	33
3.6	Adding dependencies to your package	37
3.6.1	Dependency version constraints	40
3.7	Testing your package	43
3.7.1	Writing tests	43
3.7.2	Running tests	45
3.7.3	Code coverage	48
3.8	Package documentation	49
3.8.1	Writing documentation	52
3.8.2	Writing docstrings	54
3.8.3	Creating usage examples	59
3.8.4	Building documentation	60
3.8.5	Hosting documentation online	69
3.9	Tagging a package release with version control	72
3.10	Building and distributing your package	74
3.10.1	Building your package	74
3.10.2	Publishing to TestPyPI	78
3.10.3	Publishing to PyPI	79
3.11	Summary and next steps	79
4	Package structure and distribution	83
4.1	Packaging fundamentals	83
4.2	Package structure	89
4.2.1	Package contents	89
4.2.2	Package and module names	92
4.2.3	Intra-package references	93
4.2.4	The init file	94
4.2.5	Including non-code files in a package	97
4.2.6	Including data in a package	98
4.2.7	The source layout	101
4.3	Package distribution and installation	103
4.3.1	Package installation	104
4.3.2	Building sdist and wheels	107
4.3.3	Packaging tools	108
4.3.4	Package repositories	109
4.4	Version control	110
5	Testing	111
5.1	Testing workflow	111
5.2	Test structure	114
5.3	Writing tests	116
5.3.1	Unit tests	116
5.3.2	Test that a specific error is raised	120

5.3.3	Integration tests	123
5.3.4	Regression tests	124
5.3.5	How many tests should you write	126
5.4	Advanced testing methods	126
5.4.1	Fixtures	127
5.4.2	Parameterizations	129
5.5	Code coverage	132
5.5.1	Line coverage	133
5.5.2	Branch coverage	134
5.5.3	Calculating coverage	134
5.5.4	Coverage reports	136
5.6	Version control	138
6	Documentation	139
6.1	Documentation content and workflow	139
6.2	Writing documentation	143
6.2.1	README	144
6.2.2	License	145
6.2.3	Contributing guidelines	146
6.2.4	Code of conduct	146
6.2.5	Changelog	147
6.2.6	Examples	148
6.2.7	Docstrings	150
6.2.8	Application programming interface (API) reference . .	153
6.2.9	Other package documentation	153
6.3	Building documentation	154
6.4	Hosting documentation online	162
7	Releasing and versioning	165
7.1	Version numbering	165
7.2	Version bumping	168
7.2.1	Manual version bumping	168
7.2.2	Automatic version bumping	169
7.3	Checklist for releasing a new package version	172
7.3.1	Step 1: make changes to package source files	173
7.3.2	Step 2: document your changes	173
7.3.3	Step 3: bump version number	174
7.3.4	Step 4: run tests and build documentation	176
7.3.5	Step 5: tag a release with version control	177
7.3.6	Step 6: build and release package to PyPI	178
7.4	Automating releases	181
7.5	Breaking changes and deprecating package functionality . .	181
7.6	Updating dependency versions	183

8	Continuous integration and deployment	185
8.1	An introduction to CI/CD	185
8.2	CI/CD tools	186
8.3	Introduction to GitHub Actions	187
8.3.1	Key concepts	187
8.3.2	A toy example	188
8.3.3	Actions and commands	192
8.4	Setting up continuous integration	193
8.4.1	Setup	193
8.4.2	Running tests	196
8.4.3	Recording code coverage	196
8.4.4	Build documentation	197
8.4.5	Testing continuous integration	198
8.5	Setting up continuous deployment	200
8.5.1	Setup	201
8.5.2	Automatically creating a new package version	204
8.5.3	Uploading to TestPyPI and PyPI	207
8.5.4	Testing continuous deployment	210
8.6	Summary	214
	Bibliography	219
	Index	221

List of Figures

2.1	Installing the Python extension in Visual Studio Code.	7
2.2	Executing a simple Python file called hello-world.py from the integrated terminal in Visual Studio Code.	8
2.3	Executing a simple Python file called hello-world.py from a terminal in JupyterLab.	9
2.4	Executing Python code in the RStudio.	11
3.1	Creating a new repository in GitHub.	28
3.2	Setting up a new repository in GitHub.	29
3.3	Instructions on how to link local and remote version control repositories.	30
3.4	Example figure created from the plotting function.	38
3.5	Rendered version of README.md.	54
3.6	A simple Jupyter Notebook using code from pycounts.	61
3.7	First half of Jupyter Notebook demonstrating an example workflow using the pycounts package.	61
3.8	Second half of Jupyter Notebook demonstrating an example workflow using the pycounts package.	62
3.9	The documentation homepage generated by sphinx.	63
3.10	Documentation for the pycounts plotting module.	69
3.11	Jupyter Notebook example rendered into pycounts’s documentation.	70
3.12	Tag of v0.1.0 of pycounts on GitHub.	73
3.13	Making a release of v0.1.0 of pycounts on GitHub.	74
3.14	Release of v0.1.0 of pycounts on GitHub.	75
4.1	Keep package names meaningful, memorable, and manageable.	93
4.2	The Python package cycle.	104
5.1	The testing workflow.	113
5.2	HTML test report.	137
5.3	Detailed view of the datasets module in the HTML report.	137
6.1	Example of HTML documentation generated by sphinx.	142
6.2	First half of Jupyter Notebook demonstrating an example workflow using the pycounts package.	150

6.3	Second half of Jupyter Notebook demonstrating an example workflow using the pycounts package.	151
6.4	API reference for the pycounts package.	154
6.5	API reference for the pycounts.plotting package.	155
6.6	The documentation homepage generated by sphinx.	157
6.7	Jupyter Notebook example rendered into pycounts's documentation.	161
7.1	Tag of v0.2.0 of pycounts on GitHub.	178
7.2	Release v0.2.0 of pycounts on GitHub.	179
8.1	Setting up our first GitHub Actions workflow.	188
8.2	Our first GitHub Actions workflow.	190
8.3	The logs of our first GitHub Actions workflow.	191
8.4	Example of the Codecov dashboard linked to a repository called pypkgs. Coverage decreased significantly after the most recent commit.	198
8.5	Successfully run continuous integration workflow on GitHub.	200
8.6	Continuous integration workflow logs.	201
8.7	Adding the TestPyPI API token to our GitHub repository.	208
8.8	Adding the PyPI API token to our GitHub repository.	209
8.9	Continuous deployment workflow on GitHub.	212
8.10	Successfully run continuous deployment workflow on GitHub.	213
8.11	The Python semantic release tool automatically bumped the package version from 0.2.0 to 0.3.0.	214
8.12	The Python semantic release tool automatically updated the changelog and added an entry for v0.3.0 based on commit messages.	215
8.13	The Python semantic release tool automatically created tagged release v0.3.0.	216
8.14	Deployment of new package version 0.3.0 to PyPI.	217

List of Tables

0.1	Concepts this book assumes readers have basic familiarity with.	xvi
3.1	A description of the py-pkgs-cookiecutter template prompts. .	24
3.2	A description of the tables in the pyproject.toml.	34
3.3	Typical Python package documentation.	50
4.1	Demonstration of absolute and relative intra-package imports.	94
6.1	Typical Python package documentation.	139
7.1	Examples of major, minor, and patch releases of Python. . . .	167
8.1	Terminology used in GitHub Actions.	187
8.2	Description of Python Semantic Release configuration options.	205



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Preface

Python packages are the fundamental units of shareable code in Python. Packages make it easy to organize, reuse, and maintain your code, as well as share it between projects, with your colleagues, and with the wider Python community. *Python Packages* is an open-source book that describes modern and efficient workflows for creating Python packages. The focus of this book is overwhelmingly practical; we will demonstrate methods and tools you can use to develop and maintain packages quickly, reproducibly, and with as much automation as possible — so you can focus on writing and sharing code!

Why read this book?

Despite their importance, packages can be difficult to understand and cumbersome to create for beginners and seasoned developers alike. This book aims to describe the packaging process at an accessible and practical level for data scientists, developers, and programmers. Along the way, we'll develop a real Python package and will explore all the key elements of Python packaging, including: creating a package file and directory structure, when and why to write tests and documentation, and how to maintain and update your package with the help of automated continuous integration and continuous deployment (CI/CD) pipelines.

By reading this book, you will:

- Understand what Python packages are, and when and why you should use them.
- Be able to build your own Python package from scratch.
- Learn how to document your Python code and packages.
- Write software tests for your code and automate them.
- Learn how to release your package on the Python Package Index (PyPI) and discover best practices for updating and versioning your code.
- Implement CI/CD workflows to build, test, and deploy your package automatically.
- Get tips on Python coding style, best-practice packaging workflows, and other useful development tools.

Structure of the book

Chapter 1: Introduction first gives a brief introduction to packages in Python and why you should know how to make them.

Chapter 2: System setup describes how to set up your development environment to develop packages and follow the examples in this book.

In **Chapter 3: How to package a Python**, we develop an example package from beginning-to-end as a practical demonstration of the key steps involved in the packaging process. This chapter forms the foundation of the book and will act as a reference sheet for readers creating packages in the future.

The remaining chapters then go into more detail about each step in this process, organized roughly in their order in the workflow:

- **Chapter 4: Package structure and distribution**
 - **Chapter 5: Testing**
 - **Chapter 6: Documentation**
 - **Chapter 7: Releasing and versioning**
 - **Chapter 8: Continuous integration and deployment**
-

Assumptions

While this book aims to introduce Python packaging at a beginner level, we assume readers have basic familiarity with the concepts listed in [Table 0.1](#):

TABLE 0.1: Concepts this book assumes readers have basic familiarity with.

Item	Learning resources
How to import Python packages with the <code>import</code> statement	Python documentation
How to write conditionals (<code>if/elif/else</code>) and loops (<code>for</code>)	Python documentation
How to use and write Python functions	<i>Plotting and Programming in Python: Writing Functions</i> (The Carpentries, 2021)
(Optional) Basic familiarity with version control and Git and GitHub (or similar tools)	<i>Happy Git and GitHub for the useR</i> (Bryan et al., 2021) or <i>Research Software Engineering with Python</i> (Irving et al., 2021)

Conventions

Throughout this book we use `foo()` to refer to functions, `bar` for inline commands/variables/function parameters/package names, and `__init__.py` and `src/` to refer to files and directories respectively.

Commands entered at the command line appear as below, with `$` indicating the command prompt:

```
$ mkdir my-first-package
$ cd my-first-package
$ python
```

Code entered in a Python interpreter looks like this:

```
>>> import math
>>> round(math.pi, 3)
```

```
3.142
```

Code blocks appear as below:

```
def is_even(n):
    """Check if n is even."""
    if n % 2 == 0:
        return True
    else:
        return False
```

If you are reading an electronic version of the book, e.g., <https://py-pkgs.org>, all code is rendered so that you can easily copy and paste directly from your browser to your Python interpreter or editor.

Persistence

The Python software ecosystem is constantly evolving. While we aim to make the packaging workflows and concepts discussed in this book tool-agnostic, the tools we do use in the book may have been updated by the time you read it. If the maintainers of these tools are doing the right thing by documenting,

versioning, and properly deprecating their code (we'll explore these concepts ourselves in **Chapter 7: Releasing and versioning**), then it should be straightforward to adapt any outdated code in the book.

Colophon

This book was written in JupyterLab¹ and compiled using Jupyter Book². The source is hosted on GitHub³ and is deployed online at <https://py-pkgs.org> with Netlify⁴.

Acknowledgments

We'd like to thank everyone that has contributed to the development of *Python Packages*. This is an open source book that began as supplementary material for the University of British Columbia's Master of Data Science program and was subsequently developed openly on GitHub where it has been read, revised, and supported by many students, educators, practitioners and hobbyists. Without you all, this book wouldn't be nearly as good as it is, and we are deeply grateful. A special thanks to those who have contributed to or provided feedback on the text via GitHub (in alphabetical order of GitHub username): benjy765, Carreau, chendaniely, dcslagel, eliasdabbas, fegue, firasm, Midnigher, mtkerber, NickleDave, SamEdwardes, tarensanders.

The scope and intent of this book was inspired by the fantastic *R Packages*⁵ (Wickham and Bryan, 2015) book written by Hadley Wickham and Jenny Bryan, a book that has been a significant resource for the R community over the years. We hope that *Python Packages* will eventually play a similar role in the Python community.

¹<https://jupyterlab.readthedocs.io/en/stable/index.html>

²<https://jupyterbook.org/intro.html>

³<https://github.com/UBC-MDS/py-pkgs>

⁴<https://www.netlify.com/>

⁵<https://r-pkgs.org>

About the authors

Tomas Beuzen is a data scientist and educator based in Sydney, Australia. He has a background in coastal engineering and climate science and was a teaching fellow in the Master of Data Science program (Vancouver Option) at the University of British Columbia. Tomas currently works as a data scientist in the renewable energy sector and enjoys spending his free time developing open-source, educational data science material, and using data science to solve problems in the natural and engineered world.

Tiffany Timbers is an Assistant Professor of Teaching in the Department of Statistics and a Co-Director for the Master of Data Science program (Vancouver Option) at the University of British Columbia. In these roles she teaches and develops curriculum around the responsible application of data science to solve real-world problems. One of her favorite courses she teaches is a graduate course on collaborative software development, which focuses on teaching how to create R and Python packages using modern tools and workflows.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

1

Introduction

Python packages are a core element of the Python programming language and are how you write reusable and shareable code in Python. This book assumes that readers are familiar with how to install a package using a package installer like `pip` or `conda`, and how to import and use it with the help of the `import` statement in Python.

For example, the command below uses `pip` to install `numpy` ([Harris et al., 2020](#)), the core scientific computing package for Python:

```
$ pip install numpy
```

Once the package is installed, it can be used in a Python interpreter. For example, to round `pi` to three decimal places:

```
$ python
```

```
>>> import numpy as np
>>> np.round(np.pi, decimals=3)
```

```
3.142
```

At a minimum, a package bundles together code (such as functions, classes, variables, or scripts) so that it can be easily reused across different projects. However, packages are typically also supported by extra content such as documentation and tests, which become exponentially more important if you wish to share your package with others.

As of January 2022, there are over 350,000 packages available on the Python Package Index (PyPI)¹, the official online software repository for Python. Packages are a key reason why Python is such a powerful and widely used programming language. The chances are that someone has already solved a problem that you're working on, and you can benefit from their work by downloading and installing their package. Put simply, packages are how you make

¹<https://pypi.org>

it as easy as possible to use, maintain, share, and collaborate on Python code with others, whether they be your friends, work colleagues, the world, or your future self!

Even if you never intend to share your code with others, making packages will ultimately save you time. Packages make it significantly easier for you to reuse and maintain your code within a project and across different projects. After programming for some time, most people will eventually reach a point where they want to reuse code from one project in another. For beginners, in particular, this is something often accomplished by copying-and-pasting existing code into the new project. Despite being inefficient, this practice also makes it difficult to improve and maintain your code across projects. Creating a simple Python package will solve these problems.

Regardless of your motivation, the goal of this book is to show you how to easily develop Python packages. The focus is overwhelmingly practical — we will leverage modern methods and tools to develop and maintain packages efficiently, reproducibly, and with as much automation as possible, so you can focus on writing and sharing code. Along the way, we'll also enlighten some interesting and relevant lower-level details of Python packaging and the Python programming language.

1.1 Why you should create packages

There are many reasons why you should develop Python packages!

- To effectively share your code with others.
- They save you time. Even if you don't intend to share your package with others, they help you easily reuse and maintain your code across multiple projects.
- They force you to organize and document your code, such that it can be easily understood and used at a later time.
- They isolate dependencies for your code and improve its reproducibility.
- They are a good way to practice writing good code.
- Packages can be used to effectively bundle up reproducible data analysis and programming projects.
- Finally, developing and distributing packages supports the Python ecosystem and other Python users who can benefit from your work.

References

Abbott, E. A. (1884). Flatland. Seeley and Co.

Bryan, J. , Hester, J. , and Assistants, S. T. (2021). Happy git and GitHub for the user. <https://happygitwithr.com/>.

Fucci, D. , Erdogmus, H. , Turhan, B. , Oivo, M. , and Juristo, N. (2016). A dissection of the test-driven development process: Does it really matter to test-first or to test-last? IEEE Transactions on Software Engineering, 43(7):597–614.

Harris, C. R. , Millman, K. J. , van der Walt, S. J. , Gommers, R. , Virtanen, P. , Cournapeau, D. , Wieser, E. , Taylor, J. , Berg, S. , Smith, N. J. , Kern, R. , Picus, M. , Hoyer, S. , van Kerkwijk, M. H. , Brett, M. , Haldane, A. , del Río, J. F. , Wiebe, M. , Peterson, P. , Gérard-Marchant, P. , Sheppard, K. , Reddy, T. , Weckesser, W. , Abbasi, H. , Gohlke, C. , and Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825):357–362.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. Computing in Science & Engineering, 9(3):90–95.

Irving, D. , Hertweck, K. , Johnston, L. , Ostblom, J. , Wickham, C. , and Wilson, G. (2021). Research Software Engineering with Python. Chapman and Hall/CRC.

Kluyver, T. , Ragan-Kelley, B. , Pérez, F. , Granger, B. E. , Bussonnier, M. , Frederic, J. , Kelley, K. , Hamrick, J. B. , Grout, J. , Corlay, S. , et al. (2016). Jupyter Notebooks-a publishing format for reproducible computational workflows., volume 2016. IOS Press.

The Carpentries (2021). Plotting and programming in Python: Writing functions. <https://swcarpentry.github.io/python-novice-gapminder/16-writing-functions/index.html>.

Wickham, H. and Bryan, J. (2015). R Packages. O'Reilly Media, Inc.